



ELECTRÓNICA DIGITAL III

CLASE 7 Módulo DAC

Ing. Emiliano Migliore

Módulo DAC: Configuración

La LPC1769 cuenta con un solo bloque conversor digital-analógico en escalera de resistencias conformando un divisor de tensión de Kelvin-Varley, el cual presenta una tasa de conversión de **10-bit** a una frecuencia máxima de conversión $f_{DAC_{max}}$ de 1MHz. El valor digital $VALUE_D$ de 10-bit que se convertirá a analógico $VALUE_A$ se almacena en los bits VALUE[15:6].

D/A Converter Register (DACR - address 0x4008 C000) bit description

Bit	Symbol	Value	Description	Reset Value
15:6	VALUE		After the selected settling time after this field is written with a new VALUE, the voltage on the AOUT pin (with respect to V_{SSA}) is $VALUE \times ((V_{REFP} - V_{REFN})/1024) + V_{REFN}$.	0

Previo a esta conversión se establece el tiempo de establecimiento t_s de la misma, dado por el bit BIAS[16]:

$$BIAS = 0 \Rightarrow t_{s(max)} = 1\mu s \rightarrow \begin{cases} I_{max} = 700\mu A \\ f_{DAC_{max}} = 1MHz \end{cases} \Rightarrow Rapidez \uparrow - Consumo \uparrow \downarrow$$

$$BIAS = 1 \Rightarrow t_{s(max)} = 2,5\mu s \rightarrow \begin{cases} I_{max} = 350\mu A \\ f_{DAC_{max}} = 400KHz \end{cases} \Rightarrow Rapidez \downarrow - Consumo \downarrow$$

D/A Converter Register (DACR - address 0x4008 C000) bit description

Bit	Symbol	Value	Description	Reset Value
16	BIAS ^[1]	0	The settling time of the DAC is 1 μ s max, and the maximum current is 700 μ A. This allows a maximum update rate of 1 MHz.	0
		1	The settling time of the DAC is 2.5 μ s and the maximum current is 350 μ A. This allows a maximum update rate of 400 kHz.	

Módulo DAC

Nota: Estos tiempos son válidos para cargas cuya capacitancia en el pin AOUT no exceda los 100pF. Un valor superior de impedancia, causara tiempos de establecimientos mayores a los especificados.

[1] The settling times noted in the description of the BIAS bit are valid for a capacitance load on the AOUT pin not exceeding 100 pF. A load impedance value greater than that value will cause settling time longer than the specified time. One or more graph(s) of load impedance vs. settling time will be included in the final data sheet.

Luego de este tiempo de estabilización seleccionado, la tensión en AOUT se determina como:

$$VALUE_A = VALUE_D \frac{(V_{REFP} - V_{REFN})}{2^{10}} + V_{REFN}$$

Sean $V_{REFP} = V_{DD} = 3,3V$ y $V_{REFN} = V_{SS} = 0V$, resulta:

$$VALUE_A = VALUE_D \frac{3,3V}{1024}$$

D/A Pin Description

Pin	Type	Description
AOUT	Output	Analog Output. After the selected settling time after the DACR is written with a new value, the voltage on this pin (with respect to V_{SSA}) is $VALUE \times ((V_{REFP} - V_{REFN})/1024) + V_{REFN}$. Remark: The DAC output is disabled when the device is in deep-sleep, power-down, or deep power-down mode.

Ejemplo 1: DAC mediante Interrupción por MATCH del TMR0

```
1  /*
2  * @brief   Generar una señal analógica de tipo rampa lineal de 0 a 3V3
3  *          a una frecuencia de 976[Hz].
4  * @note    Utilizar la funcionalidad de MATCH del pin P1.28 como base de
5  *          tiempo para la generación.
6  */
7
8  /* Directivas de Inclusión */
9  #include "LPC17xx.h"
10 #include "lpc17xx_pinsel.h"
11 #include "lpc17xx_timer.h"
12 #include "lpc17xx_dac.h"
13
14 /* Definición de Constantes y Macros */
15 #define NUM_WAVE_SAMPLES 1024          //DAC resolución
16
17 /* Definición de Variables Globales */
18 uint32_t WaveForm[NUM_WAVE_SAMPLES];
19
20 /* Prototipos de Funciones */
21 void cfgWaveForm(void);
22 void cfgGPIO(void);
23 void cfgTimer(void);
24 void cfgDAC(void);
25
26 /* Función Principal */
27 int main(void) {
28
29     cfgWaveForm();
30     cfgGPIO();
31     cfgDAC(); //Primero conf DAC
32     cfgTimer(); //Segundo conf TIMER (para que no dispare interrupciones antes de tener todo listo)
33
34     while(1) {}
35     return 0;
36 }
37
```

Ejemplo 1: DAC mediante Interrupción por MATCH del TMR0

```
38  /* Definiciones de Funciones */
39  void cfgWaveForm(void)
40  {
41      //Generación Rampa Lineal
42      for(uint16_t i = 0; i < NUM_WAVE_SAMPLES; i++)
43      {
44          WaveForm[i] = i;
45      }
46      return;
47  }
48
49  void cfgGPIO(void)
50  {
51      PINSEL_CFG_Type cfgPinMAT0_CH0;           //Configuración del PIN P1.28
52      PINSEL_CFG_Type cfgPinDAC;               //Configuración del PIN P0.26
53
54      cfgPinMAT0_CH0.Portnum = PINSEL_PORT_1;   //Selección de Puerto 1
55      cfgPinMAT0_CH0.Pinnum = PINSEL_PIN_28;    //Selección de Pin 28
56      cfgPinMAT0_CH0.Funcnum = PINSEL_FUNC_3;   //Funcionalidad de P1.28 como MAT0.0
57      cfgPinMAT0_CH0.Pinmode = PINSEL_PINMODE_TRISTATE;
58      cfgPinMAT0_CH0.OpenDrain = PINSEL_PINMODE_NORMAL;
59
60      cfgPinDAC.Portnum = PINSEL_PORT_0;        //Selección de Puerto 0
61      cfgPinDAC.Pinnum = PINSEL_PIN_26;         //Selección de Pin 26
62      cfgPinDAC.Funcnum = PINSEL_FUNC_2;        //Funcionalidad de P0.26 como AOUT
63      cfgPinDAC.Pinmode = PINSEL_PINMODE_TRISTATE;
64      cfgPinDAC.OpenDrain = PINSEL_PINMODE_NORMAL;
65
66      PINSEL_ConfigPin(&cfgPinMAT0_CH0);        //Inicialización TMR0: P1.28 (MAT0.0)
67      PINSEL_ConfigPin(&cfgPinDAC);             //Inicialización DAC: P0.26 (ADOUT)
68      return;
69  }
70
```

Ejemplo 1: DAC mediante Interrupción por MATCH del TMR0

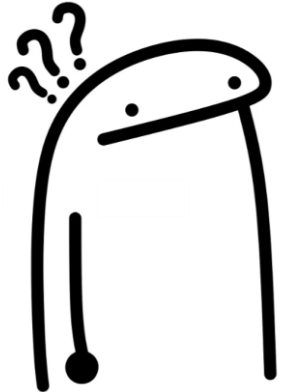
```
71 void cfgTimer(void)
72 {
73     TIM_TIMERCFG_Type    cfgTimerMode;           //Configuración de Timer en Modo Temporizador
74     TIM_MATCHCFG_Type    cfgTimerMatch;          //Configuración del Tiempo de Desborde de MATCH
75
76     cfgTimerMode.PrescaleOption = TIM_PRESCALE_USVAL; //Base de Tiempo del Prescaler a partir de valor en µs
77     cfgTimerMode.PrescaleValue  = 0;              //Base de Tiempo de Desborde de Prescaler [µs] (Time_Overflow_PC)
78
79     cfgTimerMatch.MatchChannel  = 0;              //Habilitación de Canal de MATCH0: (MR0)
80     cfgTimerMatch.MatchValue    = 1;              //Valor Máximo para TC (MR)
81     cfgTimerMatch.IntOnMatch    = ENABLE;         //Habilitación de Interrupción por MATCH: (MR0I=1)
82     cfgTimerMatch.ResetOnMatch  = ENABLE;         //Habilitación de Reset por MATCH: (MR0R=1)
83     cfgTimerMatch.StopOnMatch   = DISABLE;        //Deshabilitación de Stop por MATCH: (MR0S=0)
84     cfgTimerMatch.ExtMatchOutputType = TIM_EXTMATCH_NOTHING; //Modalidad NINGUNA en P1.28
85
86     TIM_Init(LPC_TIM0, TIM_TIMER_MODE, &cfgTimerMode); //Configuración del Modalidad del TIMER0
87     TIM_ConfigMatch(LPC_TIM0, &cfgTimerMatch);         //Configuración del TIMER0 como Temporizador
88     TIM_Cmd(LPC_TIM0, ENABLE);                         //Habilitación del TIMER0
89
90     NVIC_EnableIRQ(TIMER0_IRQn);                      //Habilitación de Interrupción por TIMER0
91     return;
92 }
```

$$f_{signal} = \frac{f_{delay}}{N_{samples}} = \frac{1}{t_{delay} N_{samples}} \Rightarrow t_{delay} = \frac{1}{f_{signal} N_{samples}}$$

$$t_{delay} = \frac{1}{976[Hz] * 1024} = 1\mu s = t_{MATCH_{TMR0}} \geq t_{s(max)} = 1\mu s (DAC)$$

Ejemplo 1: DAC mediante Interrupción por MATCH del TMR0

```
94 void cfgDAC(void)
95 {
96     DAC_Init(LPC_DAC); //Inicialización del DAC
97     //DAC_SetBias(LPC_DAC,DAC_MAX_CURRENT_700uA); //Establecimiento de Máxima Corriente de Salida en 700uA
98     //LPC_DAC->DACR &= ~(1<<16); //Establecimiento de Máxima Corriente de Salida en 700uA
99
100     return;
101 }
102
103 void TIMER0_IRQHandler(void)
104 {
105     if(TIM_GetIntStatus(LPC_TIM0, TIM_MR0_INT)){ //Consulta Interrupción por MR0 (MATCH0 del Timer0)
106         static uint16_t indexWave = 0;
107         DAC_UpdateValue(LPC_DAC, WaveForm[indexWave]); //Actualización de Valor al DAC
108         //LPC_DAC->DACR |= (WaveForm[indexWave]&0x3FF)<<6;
109         indexWave=(indexWave+1)%NUM_WAVE_SAMPLES; //Incrementa el Buffer Circular
110
111         TIM_ClearIntPending(LPC_TIM0, TIM_MR0_INT); //Limpia Bandera de Interrupción por TIMER0
112     }
113     return;
114 }
115
```



Observación:

Si el temporizador genera una interrupción cada 1[μ s] para actualizar el DAC, ¿cuántas interrupciones por segundo debe atender la CPU y qué impacto tendrá esto en su carga de procesamiento?